

**TECNOLOGICO NACIONAL DE MÉXICO
INSTITUTO TECNOLÓGICO DE TIJUANA**

**SUBDIRECCIÓN ACADÉMICA
DEPARTAMENTO DE SISTEMAS Y COMPUTACIÓN**

SEMESTRE:

Enero - Junio 2026

CARRERA:

Ingeniería en Sistemas Computacionales

MATERIA:

Patrones de diseño de software

TÍTULO ACTIVIDAD:

Patrón de Diseño Estado

UNIDAD A EVALUAR:

3 o 4

NOMBRE Y NÚMERO DE CONTROL DEL ALUMNO:

Casales Crus Josue Jacob 21211927

NOMBRE DEL MAESTRO (A):

Maribel Guerrero Luis

Código:

IState.cs

```
public interface IState  
{  
    void Handle(Estacionamiento espacio);  
    string GetNombre();  
}
```

LibreState.cs

```
using System;  
  
public class LibreState : IState  
{  
    public void Handle(Estacionamiento espacio)  
    {  
        Console.WriteLine($"Espacio {espacio.Id} ahora pasa de LIBRE a  
OCUPADO.");  
        espacio.State = new OcupadoState();  
    }  
  
    public string GetNombre() => "LIBRE";  
}
```

OcupadoState.cs

using System;

```
public class OcupadoState : IState
{
    public void Handle(Estacionamiento espacio)
    {
        Console.WriteLine($"Espacio {espacio.Id} ahora pasa de OCUPADO a LIBRE.");
        espacio.State = new LibreState();
    }

    public string GetNombre() => "OCUPADO";
}
```

ReservadoState.cs

using System;

```
public class ReservadoState : IState
{
    public void Handle(Estacionamiento espacio)
    {
        Console.WriteLine($"Espacio {espacio.Id} ahora pasa de RESERVADO a OCUPADO.");
        espacio.State = new OcupadoState();
    }
}
```

```
public string GetNombre() => "RESERVADO";  
}
```

Estacionamiento.cs

```
using System;
```

```
public class Estacionamiento  
{  
    public int Id { get; set; }  
    public IState State { get; set; }  
  
    public Estacionamiento(int id, IState state)  
    {  
        Id = id;  
        State = state;  
    }  
  
    public void SolicitarCambio()  
    {  
        State.Handle(this);  
    }  
  
    public void MostrarEstado()  
    {  
        Console.WriteLine($"Espacio {Id}: {State.GetNombre()}");  
    }  
}
```

```
}  
}
```

```
using System;  
using System.Collections.Generic;
```

```
class Program  
{  
    static void Main(string[] args)  
    {
```

```

Console.Write("¿Cuántos espacios deseas crear?: ");

int n = int.Parse(Console.ReadLine());

List<Estacionamiento> espacios = new List<Estacionamiento>();

for (int i = 1; i <= n; i++)
{
    espacios.Add(new Estacionamiento(i, new LibreState()));
}

int opcion;
do
{
    Console.WriteLine("\n--- MENÚ ---");
    Console.WriteLine("1. Ver estados");
    Console.WriteLine("2. Cambiar estado automático");
    Console.WriteLine("3. Reservar espacio");
    Console.WriteLine("0. Salir");
    Console.Write("Opción: ");
    opcion = int.Parse(Console.ReadLine());

    switch (opcion)
    {
        case 1:
            foreach (var e in espacios)
                e.MostrarEstado();
            break;
    }
}

```

case 2:

```
Console.Write("ID del espacio: ");  
int id = int.Parse(Console.ReadLine());  
espacios[id - 1].SolicitarCambio();  
break;
```

case 3:

```
Console.Write("ID del espacio a reservar: ");  
int idRes = int.Parse(Console.ReadLine());  
espacios[idRes - 1].State = new ReservadoState();  
Console.WriteLine($"Espacio {idRes} ahora está RESERVADO.");  
break;
```

```
}
```

```
} while (opcion != 0);
```

```
}
```

```
}
```

Captura de pantalla

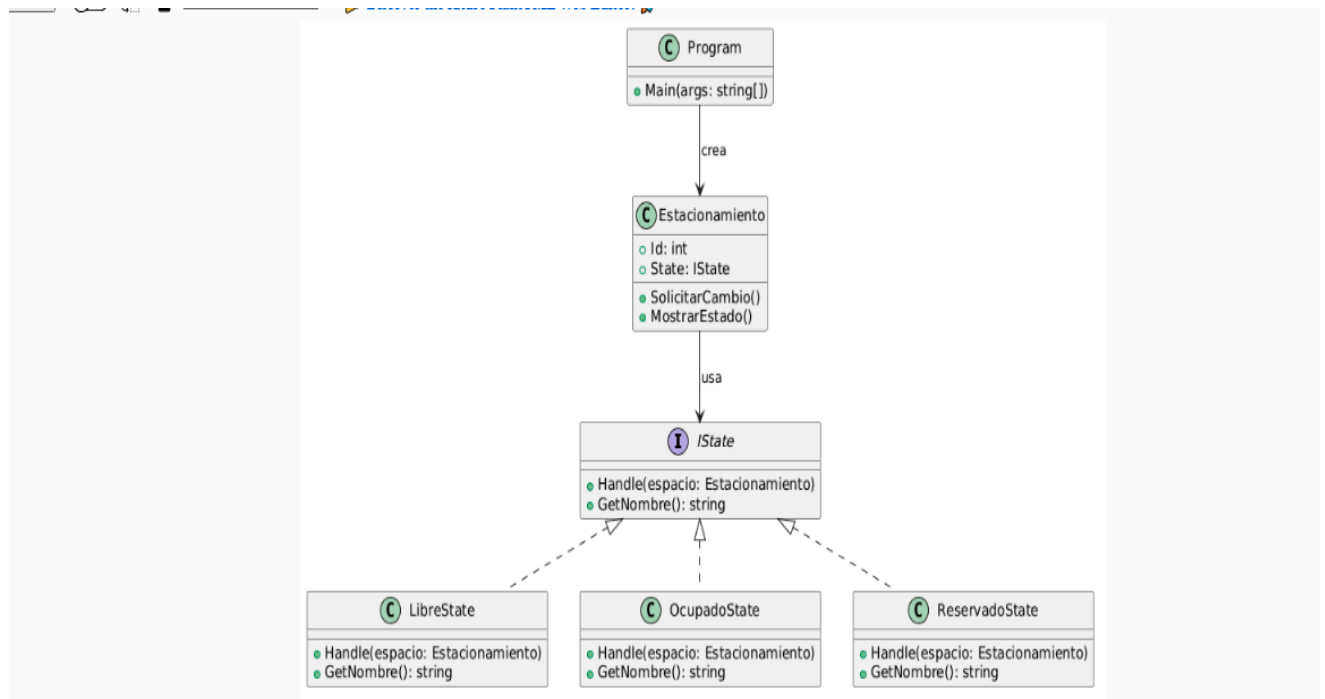
```
C:\Users\ADMIN\source\repos\Patron de Diseño Estado\Patron de Diseño Estado\bin\Debi
3. Reservar espacio
0. Salir
Opción: 2
ID del espacio: 2
Espacio 2 ahora pasa de LIBRE a OCUPADO.

--- MENÚ ---
1. Ver estados
2. Cambiar estado automático
3. Reservar espacio
0. Salir
Opción: 3
ID del espacio a reservar: 3
Espacio 3 ahora está RESERVADO.

--- MENÚ ---
1. Ver estados
2. Cambiar estado automático
3. Reservar espacio
0. Salir
Opción: 2
ID del espacio: 2
Espacio 2 ahora pasa de OCUPADO a LIBRE.

--- MENÚ ---
1. Ver estados
2. Cambiar estado automático
3. Reservar espacio
0. Salir
Opción: █
```

Diagrama uml:



Conclusiones:

Este proyecto me ayudó a comprender de mejor manera el patrón de diseño State, ya que permite que un objeto cambie su comportamiento dependiendo del estado en el que se encuentre, sin necesidad de usar múltiples condicionales como if o switch.

También pude entender cómo un sistema como un estacionamiento puede modelarse de forma más ordenada, separando cada estado en su propia clase, lo que hace que el código sea más limpio, organizado y fácil de mantener.

Al implementar el menú, el programa se vuelve más interactivo, ya que el usuario puede ver los estados de los espacios, cambiar su estado o reservarlos, lo que lo hace más parecido a un sistema real.

En general, este ejercicio me sirvió para reforzar mis conocimientos en programación orientada a objetos y para entender cómo los patrones de diseño ayudan a resolver problemas de forma más estructurada en proyectos reales.